

ReaDy: A ReRAM-Based Processing-in-Memory Accelerator for Dynamic Graph Convolutional Networks

Yu Huang¹, Long Zheng¹, *Member, IEEE*, Pengcheng Yao, Qinggang Wang, Haifeng Liu¹, Xiaofei Liao¹, *Member, IEEE*, Hai Jin¹, *Fellow, IEEE*, and Jingling Xue, *Senior Member, IEEE*

Abstract—Dynamic graph convolutional networks (DGCNs) have emerged as an effective approach to analyzing graph data that is constantly changing. The typical DGCNs incorporate not only graph convolutional networks (GCNs) to extract the structural information but also with recurrent neural networks (RNNs) to capture the temporal information from evolving graph data. These two alternative execution kernels of DGCNs impose unique architecture challenges for both types of kernels to be implemented efficiently. The presence of complex execution patterns of DGCNs renders existing architectures unsuitable. In this article, we present the first DGCN accelerator with an integrated architecture, named ReaDy, to accelerate DGCNs based on emerging PIM-featured ReRAM architectures. ReaDy is novel with an integrated architecture that enables running the GCN and RNN kernels of DGCNs simultaneously. Specifically, ReaDy is equipped with a redundancy-free scheduling mechanism to alleviate intrinsic dynamic irregularity for the GCN kernel, improving hardware utilization. In addition, ReaDy also includes a locality-aware dataflow strategy to exploit the inherent intervertex data locality for the RNN kernel, reducing superfluous data accesses to vertices and weight parameters. In a holistic view, ReaDy further enhances the entire system via an interkernel pipeline to reduce the off-chip accesses of intermediate results, boosting the overall efficiency of DGCNs significantly. Compared to the state-of-the-art software framework, PyGT, running on Intel Xeon E5-2680v4 CPU and NVIDIA Ampere A100 GPU, ReaDy achieves the average speedups of 955× and 27.33×, and the average energy savings of 1093× and 80.21×, respectively. In

addition, ReaDy outperforms ReFlip-ERA, which is obtained by combining a state-of-the-art GCN accelerator ReFlip and RNN accelerator ERA-LSTM, by an average speedup of 8.30× and an average energy saving of 7.29×.

Index Terms—Accelerator, dynamic graph convolutional network (DGCN), processing-in-memory (PIM), resistive random access memory (ReRAM) architectures.

I. INTRODUCTION

GRAPHS are widely used to represent pairwise relationships. In the real world, graphs are often subject to change with a series of graph snapshots at different timestamps [25]. A typical example is the distribution graph of coronavirus cases that may evolve constantly with its pandemic spread across the world [36]. To learn the useful information under such circumstances, dynamic graph convolutional networks (DGCNs) have emerged to enable machine learning on such dynamic graphs, which are widely used in a wide variety of application domains, such as social network analytics [31], urban traffic prediction [54], and pandemic forecasting [36].

Unlike previous learning models on static graphs [15], [27], which aim to analyze the latent information of graph structure, DGCNs are more complex as they can both extract the structure information from an individual graph snapshot and capture the temporal information across different snapshots. To achieve this goal, DGCNs [31], [36], [54] often rely on an iterative process with two basic kernels: a graph convolutional neural network (GCN) and a recurrent neural network (RNN). The GCN kernel [15], [27] aggregates the features of neighboring vertices from one single snapshot to obtain the representation features of each vertex. The RNN kernel [11], [19] captures the temporal aspects by collecting the representation features of each vertex across the snapshots.

Hardware acceleration has recently been shown to be effective in boosting the GCNs [13], [14], [22], [29], [37], [50] and RNNs [16], [17], [43], [44], [55], offering impressive performance improvements over generic processors. At the first glance, one feasible intuition for realizing DGCN models is to directly harness existing GCN and RNN accelerators to handle the GCN and RNN kernels separately. Unfortunately, DGCNs exhibit unique *dynamic irregularity* and *intervertex locality* such that applying existing GCN and RNN solutions to DGCNs cannot accelerate DGCNs effectively and efficiently.

On the one hand, due to the graph sparsity, the GCN kernel is usually both sparse in computation and irregular in memory access [50]. Therefore, the GCN accelerator family often needs

Manuscript received 2 August 2022; accepted 2 August 2022. Date of current version 24 October 2022. This work was supported by the National Natural Science Foundation of China under Grant 62072195, Grant 61832006, Grant 61825202, and Grant 61929103. This article was presented at the International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS) 2022 and appeared as part of the ESWEET-TCAD special issue. This article was recommended by Associate Editor A. K. Coskun. (*Corresponding author: Long Zheng.*)

Yu Huang, Long Zheng, Pengcheng Yao, Qinggang Wang, and Haifeng Liu are with the National Engineering Research Center for Big Data Technology and System, Service Computing Technology and System Laboratory, Cluster and Grid Computing Laboratory, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China, and also with Zhejiang Laboratory, Hangzhou 311121, China (e-mail: yuh@hust.edu.cn; longzh@hust.edu.cn; pcayao@hust.edu.cn; qgwang@hust.edu.cn; hfliu@hust.edu.cn).

Xiaofei Liao and Hai Jin are with the National Engineering Research Center for Big Data Technology and System, Service Computing Technology and System Laboratory, Cluster and Grid Computing Laboratory, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: xfliao@hust.edu.cn; hjin@hust.edu.cn).

Jingling Xue is with the School of Computer Science and Engineering, University of New South Wales, Sydney, NSW 2052, Australia (e-mail: jingling@cse.unsw.edu.au).

Digital Object Identifier 10.1109/TCAD.2022.3199152

to be accompanied by matched software co-designs to exploit the accelerator potential. For instance, HyGCN [50] adopts graph partitioning to enhance data reuse, AWB-GCN [13] uses an auto-tuning approach to address load imbalance, and ReFlip [22] reorganizes graph structure to tradeoff data movement overhead and computation efficiency. These techniques [8], [13], [22], [29], [50] are tightly coupled with a specific graph structure. However, *the graph structure in the DGCN is not only sparse but also varies dynamically*. Once the graph changes, all the changes must be processed, incurring substantial extra overhead (e.g., up to $\sim 10\times$ against the standard execution time [13]). Moreover, this overhead is hardly amortized by multiple runs of different models.

On the other hand, for the RNN accelerator family, in order to guarantee the request quality of services, the batch size is usually as small as 1–10 [52] so that each request can be responded quickly. Therefore, existing RNN accelerators [16], [17], [43], [44], [55] focus more on optimizing data dependencies between different timestamps. However, DGCNs exhibit a large number of the same data access patterns for the vertices in a timestamp. That is, *the weight parameters are fully shared by all vertices in each snapshot, showing the unique intervertex locality*. Given that the number of vertices in each graph snapshot can easily be millions [40], existing RNN accelerators [16], [17], [43], [44], [55] will miss such intervertex locality in DGCNs, resulting in superfluous data accesses to vertices and weight parameters.

Due to the complex execution patterns of DGCNs, an efficient architecture should enable both GCN and RNN kernels to be implemented efficiently. Interestingly, we observe that the GCN and RNN kernels of DGCNs have the operational similarity from the perspective of matrix-vector multiplication (MVM), which can be understood as being equivalent to a set of matrix operations [19], [27]. Recent studies [10], [21], [42] demonstrate that resistive random access memory (ReRAM) can serve as the essential compute logic to perform the MVM operation with high performance and low energy cost compared to the CMOS-based architecture. Thus, we are motivated to architect GCN and RNN kernels with an integrated ReRAM-based architecture, which is capable of performing MVM with the processing-in-memory (PIM) feature enabled to offer significant performance and energy benefits.

However, achieving this idea on ReRAM crossbar arrays poses several challenges. First, the GCN kernel of DGCNs exhibits dynamic irregularity in the sense that each vertex requires aggregating features from all its source neighbors, which vary across different graph snapshots. This can cause the hardware efficiency on ReRAM arrays to be rather low. Second, the RNN kernel in DGCNs is inherently involved with substantial data dependencies in the sense that the processing for each vertex requires the corresponding features in a previous snapshot. This may result in poor data locality with superfluous off-chip data accesses. Finally, the GCN and RNN kernels in DGCNs are highly data-dependent and are thus often executed alternatively. This may impose a large number of movement for the immediate results and further degrade the resource utilization of the underlying hardware.

In this article, we propose the first DGCN accelerator, named ReaDy, with an integrated architecture, which enables running the GCN kernel and RNN kernel on the same underlying hardware to improve the efficiency of the overall system. ReaDy features three new designs. First, we present a lightweight yet effective scheduling mechanism to reduce

the vertex feature redundancies by fully exploiting the potential temporal locality in real-world graphs [32] for the GCN kernel. Second, we propose a locality-aware dataflow execution model for the RNN kernel with parameter fusion to exploit the intervertex locality. Finally, we also orchestrate an interkernel pipeline to eliminate substantial off-chip memory access to the intermediate data with maximum computation parallelism.

The key contributions of this article are as follows.

- 1) We present the first hardware accelerator with an integrated ReRAM-based PIM architecture to enable graph learning on dynamic graphs.
- 2) We propose its tailored scheduling and dataflow designs to tackle dynamic irregularity and leverage locality in DGCNs, and then adopt the pipeline coordination to improve the efficiency of the integrated architecture.
- 3) We evaluate ReaDy with representative DGCN models on a range of graph datasets. ReaDy outperforms the state-of-the-art CPU, GPU, and the ReFlip-ERA accelerator by $955\times$, $27.33\times$, and $8.30\times$ in terms of speedups, and by $1093\times$, $80.21\times$, and $7.29\times$ in terms of energy savings.

II. BACKGROUND AND MOTIVATION

We review the background for DGCNs and ReRAM, analyze the gap between DGCNs and some related solutions, and describe our motivation behind designing a DGCN accelerator.

A. Dynamic Graph Convolutional Networks

Dynamic Graphs: Graphs are widely used to represent pairwise relationships in many applications, where entities are represented by vertices and the relationship between two entities is captured as an edge. In many real-world scenarios [31], [36], [54], graphs evolve over time, e.g., new vertices and edges are added or removed frequently. A dynamic graph can be organized in two representation forms [25]: 1) a continuous time dynamic graph (CTDG) and 2) a discrete-time dynamic graph (DTDG). In CTDG, a dynamic graph is denoted as a pair $(\mathcal{G}, \mathcal{O})$. $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a static graph representing an initial state, where $\mathcal{V}$ is the vertex set and $\mathcal{E}$ is the edge set. $\mathcal{O}$ is a set of updates for vertices and edges. In DTDG, a dynamic graph is viewed as a sequence of discrete snapshots $(\mathcal{G}^1, \mathcal{G}^2, \dots, \mathcal{G}^t)$ sampled at regular intervals. $\mathcal{G}^t$ indicates a graph snapshot at the timestamp t . In this article, we design ReaDy based on the DTDG representation, since it is more compatible with existing solutions on static graphs [7], [41].

DGCNs: Fig. 1(a) illustrates a basic DGCN inference procedure based on DTDG. In the k th iteration, the GCN kernel extracts the representation vector information for all vertices, e.g., Z^k for the snapshot $\mathcal{G}^k$. The extracted information Z^k is then fed into the RNN kernel to generate a hidden state vector H^k , which contains both graph structure and temporal information. The DGCN computation in the k th iteration can be formalized as follows:

$$\begin{aligned} Z^k &= \text{GCN}(\mathcal{G}^k) \\ H^k &= \text{RNN}(H^{k-1}, Z^k). \end{aligned} \quad (1)$$

- 1) *The GCN Kernel:* Fig. 1(b) shows a typical GCN procedure, which can be defined below

$$z_v^k = \text{ReLu}(A^k x_v^k W_{\text{com}}) \quad (2)$$

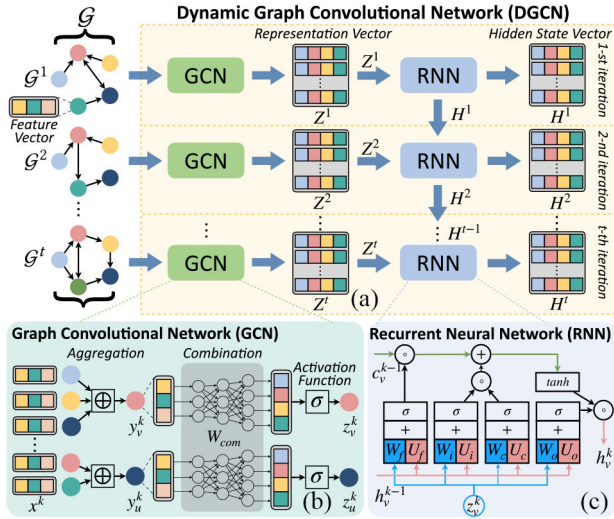


Fig. 1. Illustrative example for DGCN inference, including (a) overall processing framework, (b) typical GCN kernel, and (c) typical RNN kernel.

where A^k is the normalized Laplacian matrix over the adjacency matrix of the graph $\mathcal{G}^k$ [27], x_v^k is the initial feature vector of the vertex v at the timestamp k , and W_{com} is a combination weight parameter matrix. The GCN kernel typically contains an aggregation phase and a combination phase. In the aggregation phase, each vertex will collect localized features from its predecessor neighbors. The combination phase will then transform each vertex into a new feature based on the aggregated features.

- 2) *The RNN Kernel*: Fig. 1(c) gives a typical RNN procedure. We use the most popular long short-term memory (LSTM) [19] as an example, which can be described below

$$\begin{aligned}
 i_v^k &= \text{sigmoid}(W_{i_v} z_v^k + U_{i_v} h_v^{k-1} + b_i) \\
 f_v^k &= \text{sigmoid}(W_{f_v} z_v^k + U_{f_v} h_v^{k-1} + b_f) \\
 o_v^k &= \text{sigmoid}(W_{o_v} z_v^k + U_{o_v} h_v^{k-1} + b_o) \\
 c_v^k &= f_v^k \circ c_v^{k-1} + i_v^k \circ \tanh(W_{c_v} z_v^k + U_{c_v} h_v^{k-1} + b_c) \\
 h_v^k &= o_v^k \circ \tanh(c_v^k).
 \end{aligned} \quad (3)$$

We focus on the main components of LSTM for space-saving here. LSTM involves four input matrix multiplications by multiplying the representation vector z_v^k with four input weight matrices $W_{i,f,o,c}$. In addition, LSTM also has four hidden matrix multiplications by multiplying the hidden vector h_v^{k-1} with four hidden weight matrices $U_{i,f,o,c}$. The symbols f_v^k , o_v^k , and c_v^k indicate the forget gate, output gate, and cell state feature of vertex v at the timestamp k , respectively. In a snapshot, there are no data dependencies among the eight matrix multiplications, while across different snapshots, the hidden state at the timestamp k depends on the one at the timestamp $k-1$. LSTM also includes a few element-wise additions (+) and products ($\circ$), as well as activation functions such as sigmoid.

We note that the procedure mentioned in (1) is flexible to build a DGCN model with different GCN and RNN kernel

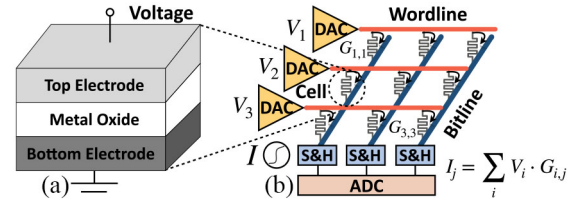


Fig. 2. ReRAM architecture with (a) structure of a cell, and (b) computational model of ReRAM crossbars.

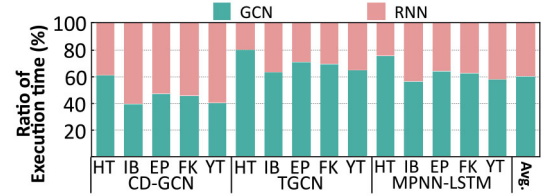


Fig. 3. Execution time breakdown of the GCN and RNN kernels in DGCNs.

implementations [31], [36], [54]. For instance, CD-GCN [31] employs a variant of GCN and LSTM for social network analysis. TGCN [54] combines the GCN with gated recurrent units (GRUs) [11] for traffic prediction. MPNN-LSTM [36] uses the message passing neural network (MPNN) [15] and LSTM for pandemic forecasting.

B. ReRAM Basics

ReRAM is an emerging nonvolatile memory that stores different data by changing the underlying resistance [20]. Fig. 2(a) depicts a ReRAM cell with a metal-insulator-metal structure [20], which has an oxide layer sandwiched between two electrodes. By applying an external voltage between the top and bottom electrodes, the resistance of a ReRAM cell can be switched between a high resistance state (HRS) and a low resistance state (LRS), which are used to represent the logic “0” and the logic “1,” respectively.

An area-efficient array organization for ReRAM is the crossbar structure as shown in Fig. 2(b), which is endowed with the PIM capability of performing in-situ MVM operations [10], [42], [45]. The cells in the crossbar array are first programmed to be conductance $G_{i,j}$, where i and j denote the number of rows and columns, respectively. The input data can be converted to analog voltages V_1 , V_2 , and V_3 applied to each wordline by applying digital-to-analog-converters (DACs). The cell (i, j) passes the current $V_i \cdot G_{i,j}$ into the bitline, based on Kirchoff’s Current Law. The accumulative output currents flowing to the end of the same bitline can be viewed as the result of the dot product operation, $I_j = \sum_i V_i \cdot G_{i,j}$, and converted to digital values using analog-to-digital-converters (ADCs). An MVM operation that is performed using a crossbar array can be completed in one cycle [20].

C. Motivation

As described in Section II-A, DGCNs consist of GCN and RNN kernels. We conduct a set of experiments to break down the execution time of DGCNs operating over a state-of-the-art DGCN software framework, PyGT [41], on NVIDIA A100 GPU. Fig. 3 gives the results on five real-world dynamic graphs, HT, IB, EP, FK, and YT, under three different models, CD-GCN, TGCN, and MPNN-LSTM. The details

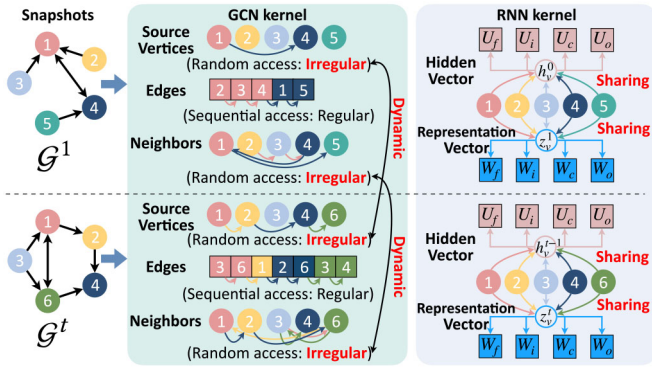


Fig. 4. Example for illustrating DGCNs.

of the system configuration and datasets used are given in Section V-A. Although different execution times are observed under different dataset and model combinations, the GCN and RNN kernels are both time-consuming computations. For instance, the percentage of the GCN kernel execution time can be up to 80.42% for TGCN on HT, but only 39.59% for CD-GCN on IB. Therefore, both kernels need to be carefully implemented in order to enable efficient execution of DGCNs.

A seemingly feasible intuition for realizing DGCN models is to directly harness existing GCN [13], [14], [22], [29], [50] and RNN [16], [17], [43], [44], [55] accelerators to process the GCN and RNN kernels separately. However, existing accelerators are not well suited to accelerating DGCNs. Fig. 4 exemplifies the execution pattern of DGCNs for two specified snapshots $\mathcal{G}^1$ and $\mathcal{G}^t$. As can be observed, the GCN kernel heavily relies on the graph structure that is inherently sparse, resulting in completely random memory accesses to a source vertex and its neighbors. For example, when traversing the neighbors of 4 in $\mathcal{G}^1$, a random access is made between 1 and 5. Moreover, the graph structure varies across the snapshots, the memory accesses to a source vertex and its neighbors are not only irregular but also dynamic. For instance, traversing 4 will generate another random access between 2 and 6 in $\mathcal{G}^t$. Existing GCN accelerators [13], [14], [22], [29], [50] include several optimization techniques that rely on preprocessing to restructure the graph organizations to alleviate the irregularity. They focus on GCN acceleration for static graphs so that the significant overhead of graph restructuring can be omitted in the critical paths. However, such high restructuring overhead (e.g., seconds in AWB-GCN [13]) is not tolerable when processing dynamic graphs in DGCNs.

On the other hand, the RNN kernel executes the MVM operation for each vertex with shared weight parameters, exhibiting intervertex locality. For example, four input weight matrices $W_{i,f,o,c}$ are widely shared between 1–5 in $\mathcal{G}^1$. Unlike the conventional RNN inferences that often operate on the text and speech [52], the RNN kernel in DGCNs acts on the whole graph snapshot. The batch size of the RNN kernel in DGCNs is usually orders of magnitude larger than that of traditional RNN inferences. This opens an opportunity for reusing weight parameters to reduce abundant memory accesses. However, existing RNN accelerators may fail to leverage such a new kind of data reuse to improve performance further. Furthermore, both GCN and RNN accelerators are typically designed to either alleviate irregularity for the GCN

kernel or tackle the data dependency for the RNN kernel, while DGCNs need both. Therefore, existing accelerators can hardly support the efficient execution of both kernels simultaneously.

Interestingly, the GCN and RNN kernels in DGCNs have an operational similarity. Let us consider the GCN kernel first. According to (2), the GCN kernel will first aggregate the features from neighboring vertices. This process can be formalized as $A^k X^k$, which is an MVM-formatted computation paradigm. Then, the aggregated features will be transformed into a new feature. This process is essentially equivalent to computing $(A^k X^k) W_{com}$, which is also an MVM-formatted computation paradigm. Consider the RNN kernel next. According to (3), we see that eight matrix multiplications are involved, with the representation vector Z^k and the hidden vector H^k multiplying the input weight matrices $W_{i,f,o,c}$ and hidden matrices $U_{i,f,o,c}$, respectively.

From the perspective of an MVM operation, it is clear that the GCN and RNN kernels can be understood as performing equivalently a set of matrix operations [19], [27]. Previous studies [10], [42], [45] have demonstrated that ReRAM is a natural fit for accelerating MVM operations with orders-of-magnitude energy efficiency improvements possible. In this work, we have devised a ReRAM-based architecture that allows the GCN and RNN kernels to run efficiently on an integrated architecture (Section III).

However, adopting DGCNs in ReRAM-based architectures is nontrivial, as the unique characteristics of DGCNs impose new requirements on architecture design. First, graphs are often dynamic and sparse, rendering ReRAM crossbars to be significantly underutilized. Thus, we are required to alleviate the dynamic irregularity in GCNs to enhance crossbar utilization. Second, we are further required to exploit the intervertex locality to maximize the data reuse. Finally, we must fuse the execution of the two main kernels to maximize efficiency in a holistic view. In this work, we propose our co-designs tailored to exploit such DGCN characteristics to improve the hardware efficiency of an integrated architecture (Section IV).

III. READY ARCHITECTURE

We first present an architecture overview of ReaDy and then introduce the workflow of the DGCNs under ReaDy.

A. Overview

Fig. 5 depicts the overall architecture of ReaDy. At a high level, ReaDy adopts a hierarchical architecture. A ReaDy chip consists of a number of processing engines (PEs) connected through a mesh-based on-chip interconnect. Each PE includes multiple computation units (CUs). ReaDy is capable of connecting with a sizeable off-chip memory for large graphs, which are sequentially streamed into PEs through a memory controller. We describe each component of ReaDy below.

Off-Chip Memory: The off-chip memory is used to store the input and output data. The input data includes a sequence of graph snapshots and weight parameters. For graph snapshots, vertex features are stored in a dense matrix format, enabling their direct mapping into ReRAM crossbars to avoid the format conversion overhead. Due to the sparsity of graph structure, edge data of each snapshot is stored in the compressed sparse column (CSC) [45] format for space-saving.

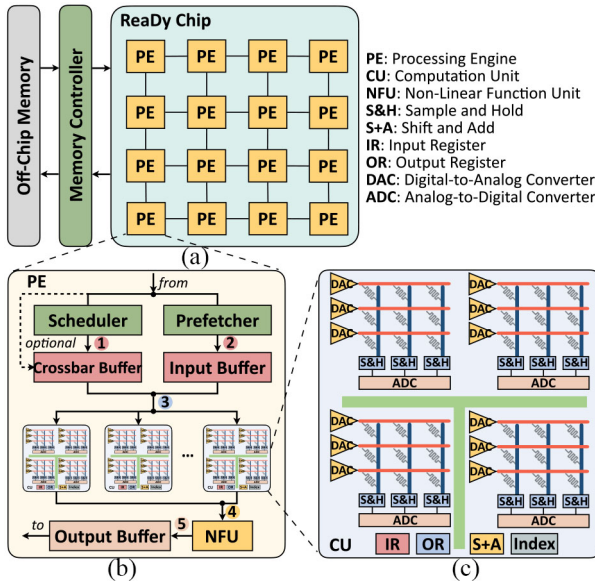


Fig. 5. Overview of the ReaDy architecture with (a) its hierarchical structure, (b) its PE, and (c) its CU microarchitecture.

As for weight parameters, they are stored in a dense matrix form in the off-chip memory and loaded into the on-chip crossbar array in a layer-wise manner. The reason behind this is that the input data for the weight parameters depends on the results of the aggregated vertex features of the graph structure. As the size of a graph is often significantly larger than the total size of crossbars, storing the weight parameters of all the layers into crossbars as in previous work [16], [42] will cause most crossbars to store other layers' weight parameters but not actually use them in the current layer. Note that, the hidden features of intermediate timestamps can be removed once their corresponding computations are done.

PE: As shown in Fig. 5(b), a PE consists of a vertex scheduler, a vertex prefetcher, a set of on-chip buffers, a nonlinear function unit (NFU), and multiple CUs. The vertex scheduler controls the scheduling order of vertices to reduce vertex data loading overhead (Section IV-A). The prefetcher prefetches the vertex data to hide the memory access latency (Section IV-C). The on-chip buffers include a crossbar buffer (for caching the data that needs to be mapped into crossbars), an input buffer (for storing the input data of crossbars temporarily), and an output buffer (for storing the results generated by CUs). To support the implementation of a nonlinear function, the NFU is also included in the PE.

Note that all PEs in ReaDy are homogenous and can thus be used alternatively for the GCN and RNN kernels of DGCNs simultaneously. ReaDy allows intermediate data induced by two kernels within each layer to pass directly to the downstream PEs via deterministic X-Y routing in a mesh network, avoiding significant off-chip data movement.

CU: A CU consists of some essential compute logics to support the efficient execution of DGCNs. Since the MVM is the dominant operation in both GCN and RNN kernels, ReaDy adopts the ReRAM crossbar as its hardware fabric, enabling computing one MVM operation in one cycle [20].

Specifically, the ReRAM crossbar arrays in CU are organized in a flat matrix layout as shown in Fig. 5(c). A DAC is connected to each wordline of a crossbar to convert its input data to a voltage. The accumulated currents on the bitlines

of the crossbar are read out by sample and hold (S&H) circuits and fed to the shared ADCs. In addition to the crossbar arrays, input and output registers (IR/OR) are used to cache the intermediate data. High computation resolution can be achieved based on low-resolution cells via shift and add (S+A) units through merging the results of adjacent columns in the crossbar under the digital signal [42], [45]. The index buffer is triggered during the GCN kernel execution to record the IDs of the vertices being mapped on the crossbar, in order to help the scheduler of the PE in identifying the next vertex to be scheduled (Section IV-A).

Controller: The memory controller manages data interactions between the off-chip memory and the ReaDy chip. Input data is prefetched by the prefetcher in PE to on-chip buffers to hide the latency of the off-chip memory accesses. Output results are written back to the off-chip memory in batches to enforce sequential memory accesses. Note that, the immediate data between the GCNs and RNNs kernels in each layer gives rise to non-off-chip data communication, and it propagates directly across the PEs via the on-chip network under the interkernel pipeline control (Section IV-C).

B. Workflow

Overall, ReaDy performs DGCNs in a snapshot-by-snapshot manner to decouple the data dependencies of different snapshots. For each snapshot, ReaDy allocates a certain number of PEs to the GCN and RNN kernels depending on their workloads. In a holistic view, both kernels are pipelined to execute simultaneously, thus avoiding the movement of intermediate data in between. To exploit the vertex-level parallelism in each snapshot, ReaDy adopts the SIMD model by assigning vertex processing workloads to different compute units.

Specifically, in the GCN kernel, the features of each vertex are first operated on the sparse graph data to collect the localized features from its predecessor neighbors. A graph is partitioned into as many contiguous vertex chunks as the number of assigned PEs, with one chunk per PE. To avoid ineffective computations, vertex features are loaded into the crossbar array, and the corresponding edges are acted as the input. The scheduler in a PE is designed to determine the scheduling order of vertices to exploit locality. After aggregating neighboring features, the aggregated vertex features are combined with the weight parameters for feature extraction. Finally, the generated results are activated by the NFU and passed to the downstream PEs for RNN kernel processing.

In the RNN kernel, there are two main computations regarding the vertex features captured by the current snapshot and the historical snapshots. For the vertex features captured in the current snapshot, the corresponding PE receives the combined vertex features produced by the upstream PE and loads the input weight matrix from the off-chip memory. Since all vertices share the same weight parameters, ReaDy loads the weight matrix into ReRAM crossbars for reusability, enabling the weight parameters to be loaded only once. As for the computation of historical vertex features, a processing pattern similar to the current features is exploited. The only difference is that the associated PE first loads its hidden features from the off-chip memory based on the vertex index processed by the upstream PE. Note that the hidden features to be loaded are determined when the scheduler sets the vertex scheduling order, whereby ReaDy leverages a prefetching mechanism to hide the off-chip memory access latency.

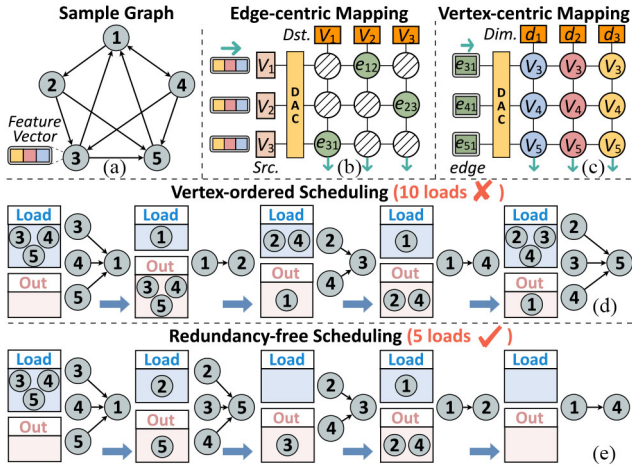


Fig. 6. Example for the GCN kernel with (a) sample graph, (b) edge-centric mapping scheme that maps sparse edge data into crossbars, (c) vertex-centric mapping that maps vertex features into crossbars, (d) index-ordered scheduling, and (e) our redundancy-free scheduling.

IV. READY DESIGNS

We describe the critical designs of ReaDy to exploit the underlying hardware fully. In particular, ReaDy customizes the vertex scheduling order to decouple dynamic sparse graph structures with regular crossbar-based ReRAM arrays during the GCN phase (Section IV-A). In addition, ReaDy also leverages the intervertex locality in each snapshot to maximize data reuse during the RNN phase (Section IV-B). Finally, ReaDy holistically improves overall efficiency by fusing GCN and RNN kernels *via* an interkernel pipeline (Section IV-C).

A. GCN Kernel: Redundancy-Free Scheduling

We use a simple graph in Fig. 6(a) to illustrate our redundancy-free scheduling for the GCN kernel. As depicted in Fig. 1(b), each vertex needs to aggregate its neighbors' features simultaneously. In an edge-centric mapping, which is widely adopted in ReRAM-based graph processing accelerators [23], [45], we must map edge data into crossbars and feed vertex data as the input of crossbars. However, real-world graphs are often highly sparse, causing the ReRAM crossbars to suffer from substantial ineffective computations on zero-valued weights as shown by the shaded circles in Fig. 6(b).

GCNs can act on graphs where each vertex can involve 100–1000 feature dimensions [50]. As shown in Fig. 6(c), ReaDy follows the vertex-centric mapping as in ReFlip [22], which maps vertex features into crossbars and feeds edge data as the input. To fit the tightly coupled crossbar structure, the input edges of a crossbar are required to point to the same destination vertex while per-row crossbar cells store the source vertex features of the underlying edge. In this way, crossbar cells can be utilized more fully so as to eliminate ineffective computations caused by sparse graph structure [45].

However, as different destination vertices are adjacent to different source vertices, the vertex data is loaded into the crossbar on demand. This may result in redundant loads for source vertices. As shown in Fig. 6(d), following a vertex index ordered scheduling, ① is loaded 3 times. Such redundant data loads can jeopardize the computational benefits of the crossbar arrays severely. Note that different destination

vertices may share the same sources, and changing the destination vertex scheduling order within a snapshot does not affect the scheduling accuracy [25]. For instance, following an improved scheduling order in Fig. 6(e), ReaDy incurs only 5 loads (without any redundancy), while the index-ordered scheduling requires 10 loads. The problem of minimizing the number of loads can be formulated as one of maximizing the number of shared source vertices of adjacent scheduled vertices

$$\text{Maximize } F(\phi) = \sum_{i=1}^{n-1} S(\phi(i), \phi(i+1)) \quad (4)$$

where ϕ is a scheduling order, $\phi(i)$ denotes the index of the i th scheduled vertex in ϕ , $S(v, u)$ is the number of shared source vertices between vertices v and u , n is the number of vertices, and $F(\phi)$ is the total number of shared source vertices under the scheduling order ϕ . This problem is equivalent to the maximum traveling salesman problem (maxTSP), which is NP-hard [47]. Some recent approximation algorithms [47] are promising, but still run in $O(n^2 \sim n^3)$. As dynamic graphs frequently change across different snapshots, applying these earlier solutions can reduce the number of redundant data loads, but their intolerable preprocessing overhead is still too high and will thus severely offset the overall benefit achieved.

Fortunately, many vertices in real-world graphs share neighbors or have dense connections to a group of vertices (e.g., community graphs [32], [49]). This enables us to simplify the process of finding the next vertex to be scheduled for an optimal scheduling order with a simple metric: *the vertex with the maximum in-degree edges among the neighbors of the currently scheduled vertex must be selected as the next vertex to be scheduled*. The reasons are twofold. First, neighboring vertices are usually more closely related to enhance the possibility of shared neighbors. Second, a vertex with the maximum in-degree edges often has more opportunities to be shared. This way, the time complexity of the scheduling order is positively correlated with the vertex degree, which can be nearly $O(1)$ and implemented easily as a vertex scheduler in a PE (as shown in Fig. 5). Fig. 6(e) illustrates a scheduling instance with such a metric. After ① is processed, ⑤ with the maximum in-degree edges will be scheduled, and so on. This scheduling order yields only 5 loads, which are minimal.

B. RNN Kernel: Locality-Aware Dataflow

We use an example in Fig. 7 to illustrate how we exploit locality in the RNN kernel. RNN operations often involve substantial data dependencies between snapshots to capture historical information. For example, the processing of a snapshot generally requires the vertex hidden features in a previous snapshot. This results in poor data reuse [52]. It is fortunate that such data dependencies exist only between snapshots. The intrasnapshot processing does not have such a dependency restriction. Unlike traditional RNN applications that often have a small batch size (usually 1–10), each snapshot in DGCNs involves a large number (up to millions) of vertices. This exhibits a significant intrasnapshot data locality. Therefore, we are motivated to propose a locality-aware dataflow execution model to reduce superfluous data accesses by exploiting the intervertex data reuse in a snapshot.

As shown in Fig. 7, the dataflow execution involves two aspects. First, for vertex features, input weight matrices $W_{i,f,o,c}$

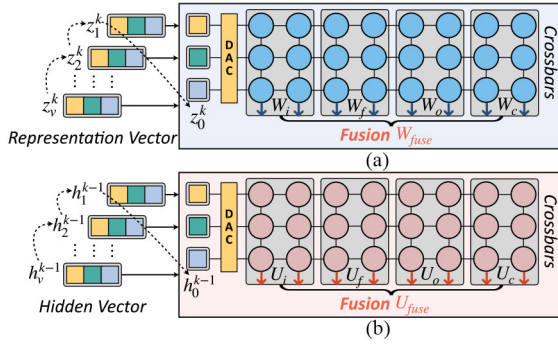


Fig. 7. Locality-aware dataflow for the RNN kernel for the eight multiplications involving (a) input weight matrices and (b) hidden weight matrices.

and hidden weight matrices $U_{i,f,o,c}$ have the same representation and hidden vector at each vertex. ReaDY fuses the corresponding input and hidden weight matrices into one large matrix W_{fuse} and U_{fuse} , respectively. This allows the weight matrices to maximize the sharing of vertex vectors, thereby improving the vertex data reuse. Second, for weight parameters, different vertices share the same weight matrix in a snapshot. ReaDY maps the fused weight matrices into crossbars and feeds vertex features as the input of crossbars. This enables loading weight matrices only once for each snapshot, enhancing the reuse of weight parameters.

C. Put All Together: Interkernel Pipeline

Dynamic graphs can evolve rapidly. The number of graph snapshots can often be easily thousands or more [41]. Thus, it is impractical to deploy all snapshots on an accelerator simultaneously. To sustain the scalability of the accelerator, ReaDY handles one DGCN snapshot at a time. Each snapshot involves a large number of vertices. This exposes opportunities for leveraging the computational parallelism of the underlying architecture. To preserve inter-PE parallelism, all PEs are performed to run the operations of a snapshot in parallel. For a snapshot, the GCN and RNN kernels follow a phase-by-phase execution: the output of the GCN kernel will be the input of the RNN kernel. ReaDY also supports the inter-PE pipeline to maximize parallelism and reduce the intermediate data movement. To be specific, we propose an interkernel pipeline to reuse the aggregated results of the GCN kernel.

However, due to irregular graphs, different vertices in the GCN kernel have different workloads. In contrast, all vertices in the RNN kernel have the same fixed execution pattern with the same amount of workload. Due to such mismatched workloads, pipeline stalls between the two kernels can be introduced. To resolve this issue, ReaDY makes use of an FIFO buffer mechanism to cache the vertex features arriving at different times from the GCN kernel and then pipeline them to the RNN kernel. As a result, the execution of GCN and RNN kernels can be decoupled, allowing us to design an interkernel pipeline separately for preserving efficiency.

Fig. 8 shows an instance of the DGCN pipeline in ReaDY. The PEs are assigned to different kernels. For example, PE#0–#3 are assigned to the aggregation operation of the GCN kernel, PE#4 to the combination phase of the GCN kernel, and PE#5 and PE#6 to the input and hidden weight multiplication of the RNN kernel, respectively. The first completed aggregated computation of ⑦ among PE#0–#3 will flow into

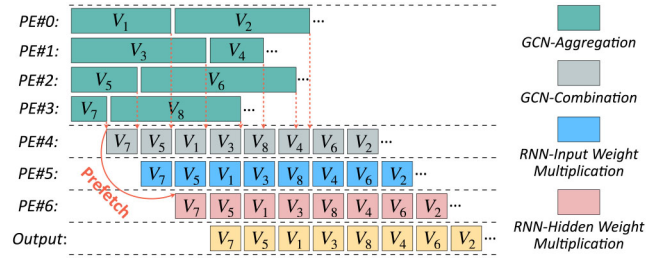


Fig. 8. Interkernel pipeline with seven PEs assigned.

an idle pipeline for the subsequent processing. Moreover, since the hidden weight multiplication of the RNN kernel needs the historical vertex feature data, the prefetcher will be triggered to issue a memory request once the aggregation operation is finished to hide the off-chip memory access latency.

To allocate PEs to each DGCN phase, we propose an approximated estimation based on the load amount

$$R_{\text{gcn-aggr}} : R_{\text{gcn-com}} : R_{\text{rnn-input}} : R_{\text{rnn-hidden}} = m \times \bar{d}_{in}(1 + \varepsilon) : \|W_{\text{com}}\| : \|W_{\text{fuse}}\| : \|U_{\text{fuse}}\| \quad (5)$$

where m is the vertex feature dimension, $\bar{d}_{in}$ is the average in-degree of a graph, $\|W\|$ is the number of weight parameters, and ε is a heuristic parameter that indicates the data load overhead of the GCN kernel. In this work, $\varepsilon = 4$ is set empirically, representing a sweet spot for yielding good results.

Note that the data load overhead can be significantly reduced by the redundant-free scheduling in Section IV-A. Also, ReaDY assigns the GCN kernel in different snapshots with different PEs in a round-robin manner. This achieves the inter-PE wear leveling, improving the endurance of ReaDY. Given over 10^{12} ReRAM write endurance [10], ReaDY is capable of achieving millions of DGCN inferences at least.

V. EVALUATION

First, we introduce our experimental setup (Section V-A). Next, we compare ReaDY with the state-of-the-art software and hardware solutions (Section V-B). Finally, we give a detailed analysis and sensitive exploration (Section V-C–V-D).

A. Experimental Setup

ReaDY Settings: We build a cycle-accurate simulator to model the on-chip microarchitectural behaviors of ReaDY, which incorporates the functionality, timing, and power models of all components in ReaDY. The off-chip memory is configured as high bandwidth memory (HBM) with 128 GB capacity and 512 GB/s bandwidth. We use Ramulator [26] to simulate timing behaviors of memory accesses to the HBM. Energy consumption is set as 7 pJ per bit as in [34].

Table I summarizes the ReaDY configuration and their power/area values. Note that the power and area values for each component are obtained according to the corresponding parameters specification, instead of the single component values. ReaDY consists of 64 PEs. Each PE has 8 CUs. In a PE, we use CACTI 6.5 [28] to model the area, power, and access latency for all inherent buffers. The energy and area for the NFU are adapted from the analysis in ISAAC [42]. The scheduler and prefetcher are implemented in Verilog RTL

TABLE I
READY CONFIGURATION

Component	Params.	Spec.	Power	Area (mm ²)
CU Properties (8 CUs per PE)				
ADC	resolution	8 bits	64 mW	0.0384
	number	32		
DAC	resolution	1 bit	16 mW	0.00068
	number	32 × 128		
Crossbar Array	bits per cell	2 bits	9.6 mW	0.0008
	size	128 × 128		
	number	4 × 8		
S&H	number	32 × 128	40 μW	0.00016
S+A	number	32	1.6 mW	0.0008
IR	size	8 KB	3.04 mW	0.0062
OR	size	1 KB	0.48 mW	0.0019
Index	size	512 B	0.23 mW	0.0008
PE Properties (64 PEs per Chip)				
CU Total	number	8	759.92 mW	0.397
CB	size	64 KB	17.66 mW	0.0086
IB	size	8 KB	1.91 mW	0.0054
OB	size	8 KB	1.91 mW	0.0054
NFU	Total	8	2.04 mW	0.0024
Scheduler & Prefetcher	-	-	58.07 mW	0.084
Chip Properties				
PE Total	number	64	53.86 W	32.18
Controller	-	-	580.41 mW	2.65
Chip Total	-	-	54.44 W	34.83

TABLE II
GRAPH DATASETS

	#Vertex	#Timestamps	#Edges	Description
HepTh (HT)	22,908	219	2,673,133	Citation Network
Imdb (IB)	150,545	28	296,188	Rating Graph
Epinions (EP)	755,760	501	13,668,320	Rating Graph
Flickr (FK)	2,302,925	134	33,140,017	Social Network
Youtube (YT)	3,223,589	203	12,223,774	Sharing Graph

and synthesized with the Synopsys toolchain using the TMS320 65-nm standard library. Each CU includes 32 ReRAM crossbars. Each crossbar with the size 128×128 is based on TaOx ReRAM cells, where the read and write latencies are 29.31 ns and 50.88 ns [33], respectively. We conservatively set the 2-bit multilevel cell to alleviate the sensing pressure [22]. The ADCs and DACs are configured to be the 8-bits and 1-bit resolution, respectively. Their area and energy consumption estimates are determined according to [42].

Datasets and Benchmarks: Table II gives the five most widely used dynamic graphs for DGCNs from network data repository [40]. We consider the three popular DGCN models: 1) **CD-GCN** [31]; 2) **TGCN** [54]; and 3) **MPNN-LSTM** [36], discussed in Section II-A. By default, each of these DGCN models is configured with 32 hidden dimensions [41]. We adopt the 32-bit fixed point representation, which is sufficient to preserve the inference accuracy [25].

Baselines: We compare ReaDy with the state-of-the-art DGCNs software framework, PyGT [41], on Intel E5-2680v4 CPU and NVIDIA A100 GPU platforms, identified as **PyGT-CPU** and **PyGT-GPU**, respectively. Their hardware specifications are shown in Table III. Intel Product Specifications [1] are used to estimate the CPU energy consumption. The GPU energy is obtained by the NVIDIA System Management Interface (nvidia-smi) [2]. Since DGCNs mainly

TABLE III
CPU AND GPU SPECIFICATIONS

CPU	Intel Xeon CPU E5-2680 v4
CPU Cores	14-Cores Per Socket, Dual Socket, 2.40GHz
Cache Per Core	L1: 64KB; L2: 256KB; L3: 2.5MB
Main Memory	256GB DDR4
GPU	Nvidia Tesla A100
GPU Cores	108 SMs, 64 CUDA cores Per SM, 1.41GHz
Cache	164KB L1 cache per SM, 40MB L2 cache
GPU Main Memory	40GB HBM2

evolve two basic kernels, GCN and RNN, we also compare ReaDy with the state-of-the-art ReRAM-based GCN accelerator ReFlip [22] and RNN accelerator ERA-LSTM [16]. To maximize execution efficiency, ReFlip and ERA-LSTM are combined into a tandem-engine architecture, denoted as **ReFlip-ERA**, for supporting GCN and RNN kernels, respectively. To make an apple-to-apple comparison, we allocate the same computational resources to ReFlip (ERA-LSTM) as those allocated to ReaDy for processing the GCN (RNN) kernel. Note that all performance and energy results are obtained by accounting for both on-chip computations and off-chip communications.

B. Overall Results

We first compare ReaDy and other baselines in terms of the overall performance and energy efficiency achieved.

Performance: Fig. 9 depicts the performance results of ReaDy against PyGT-CPU, PyGT-GPU, and ReFlip-ERA.

- 1) **ReaDy Versus PyGT-CPU:** Overall, ReaDy outperforms PyGT-CPU by $429\times \sim 1778\times$ ($955\times$ on average) owing to its highly parallel in-situ MVM operations in ReRAM crossbars and the specialized hardware design. Specifically, the benefits obtained for DGCNs are highly related to models. TGCN shows the smallest benefits (with $646\times$ on average). Unlike LSTM in other DGCN models, TGCN adopts GRU to capture the temporal information with a simpler model structure, sacrificing lots of parallelism in the RNN kernel. Thus, TGCN offers less parallelism that can be exploited by crossbars. In contrast, CD-GCN incorporates the skip-connection in the GCN kernel by concatenating the input features to the output, exposing higher parallelism that can be leveraged. Therefore, CD-GCN shows $1370\times$ performance improvement on average with the largest speedups.
- 2) **ReaDy Versus PyGT-GPU:** Compared with PyGT-GPU, ReaDy is $18.66\times \sim 46.36\times$ ($27.33\times$ on average) faster. Despite the fact that the GPU is equipped with a large number of CUDA cores (6,912) and HBM, the existence of irregular memory accesses and data dependencies in the GCN and RNN kernel make it difficult to mobilize the massive parallelism. In contrast, ReaDy addresses the above issues effectively by exploring the inter- and inner-vertex locality in DGCNs, which enables it to take full advantage of ReRAM's in-situ processing capacity with considerable execution efficiency. In particular, the benefits for DGCNs are highly related to the graph structure. For instance, HT showcases the largest speedup (up to $46.36\times$), since it has the highest average vertex degree. This implies that the aggregation phase of the GCN kernel has a large workload with a lot of irregular memory accesses, which are unfriendly for

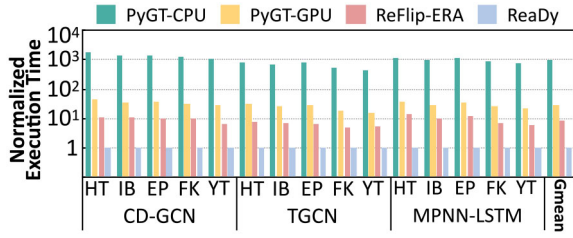


Fig. 9. Execution times of ReaDy against the state-of-the-art DGCN solutions. All results are normalized to ReaDy.

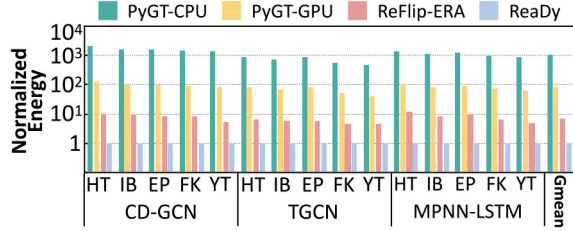


Fig. 10. Energy consumption of ReaDy against the state-of-the-art DGCN solutions. All results are normalized to ReaDy.

their processing on GPU cores, while the in-situ PIM capability of ReaDy avoids this issue naturally.

- 3) *ReaDy Versus ReFlip-ERA*: ReFlip-ERA is inferior to ReaDy by $5.17\times \sim 14.15\times$ ($8.30\times$ on average). The reasons are threefold. First, ReFlip needs to reorganize the graph structure to ensure sequential memory accesses for the GCN kernel. Since the structure of a dynamic graph varies constantly, such graph restructuring overhead accounts for 41% of the overall execution time. In contrast, ReaDy runs it directly without any preprocessing overhead. Second, ERA-LSTM uses a tile-based spatial architecture dedicated to optimizing dataflow between layers. However, this approach usually works poorly for DGCNs, because the input of the next layer of the RNN kernel depends not only on the output of the previous layer of the RNN kernel but also on the output of the GCN kernel. To ensure data consistency in DGCNs, the next layer of the RNN kernel cannot be executed until the previous layer of the GCN kernel is completed, which inevitably introduces up to 63% resource stalls. Finally, data communication between the two kernels in DGCNs is not considered in ReFlip-ERA, causing a large amount of intermediate data movement (Section V-C).

Energy Savings: Fig. 10 gives the energy-saving results. Thanks to the in-situ PIM capability of ReRAMs and custom designs tailored primarily for DGCNs, ReaDy saves $454\times \sim 2159\times$ ($1093\times$ on average) energy against PyGT-CPU. Compared with PyGT-GPU, besides data movement, massive cores deployed in GPU consume more than $3\times$ power than ReaDy. The actual power of ReaDy is only 54.44 Watt in total, while the actual power of the NVIDIA A100 often exceeds 150 Watt. Therefore, ReaDy consumes $42.66\times \sim 126.71\times$ ($80.21\times$ on average) less energy than PyGT-GPU. ReaDy is superior to ReFlip-ERA with $5.08\times \sim 11.63\times$ ($7.29\times$ on average) less energy consumption. Although both ReFlip and ERA-LSTM adopt specialized hardware units to reduce the data signal conversion overhead, ReaDy offers considerable benefits. The reasons are twofold.

First, a substantial amount of data movement has been eliminated by our redundancy-free scheduling and locality-aware dataflow execution. Second, PEs can be utilized more fully via a coordinated pipeline design.

Power and Area: As shown in Table I, the total power and area of ReaDy are only 54.44 Watt and 34.83 mm^2 , respectively. The crossbar arrays take up 9.02% and 1.17% of chip power and area. The ADCs consume most of power (60.84%) and area (56.45%). On the one hand, the energy scaling of ADCs exhibits $2\times$ improvement every 2.6 years [24]. On the other hand, several recent research efforts [22], [30] adopt analog buffers to reduce the number of data signal conversions. Therefore, the power and area efficiency of ReaDy will be further enhanced with more advances on the ADC process and fewer data conversions. The on-chip buffers occupy 6.07% and 16.65% of chip power and area. Other supporting digital units (such as S+A, NFU, and scheduler) take up small amounts of power and area at only 8.57% (power) and 17.05% (area).

C. Effectiveness of ReaDy's Designs

We study the effectiveness of ReaDy's designs: redundancy-free scheduling (Section IV-A), locality-aware dataflow (Section IV-B), and interkernel pipeline (Section IV-C).

Redundancy-Free Scheduling: Fig. 11(a) breaks down the execution time of ReaDy for the GCN kernel by using two different scheduling mechanisms: 1) vertex index ordered scheduling (ReaDy-IO) and 2) our redundancy-free scheduling. ReaDy outperforms ReaDy-IO by $2.87\times$ on average. The performance variance arises mainly from the data loading overhead. ReaDy carries a less amount of crossbar loading, since it avoids repeatedly loading the same vertex features into the crossbar. As shown in Fig. 11(b), such benefits obtained from the redundancy-free scheduling order are highly related to the graph structure. HT has the highest average vertex degree, offering more connections that can be exploited. Therefore, the number of vertex features loaded for HT is reduced by $6.83\times$, yielding up to $3.82\times$ performance gain.

Locality-Aware Dataflow: Fig. 12 compares ReaDy and ReaDy-WF for the RNN kernel, where ReaDy-WF is a variant of ReaDy without the weight matrix fusion so that vertex feature data needs to be read repeatedly for each weight matrix. ReaDy offers a $2.28\times$ speedup due to the enhanced data locality. In particular, the performance gains among the different models exhibit significant differences. ReaDy reaps the most gains (with $2.67\times$ on average) for CD-GCN. By concatenating the vertex input features to the output features of the GCN kernel, CD-GCN yields a larger amount of input vertex feature data for the RNN kernel than TGCN and MPNN-LSTM. Without fusing weight matrices, such data needs to be read four times repeatedly. In contrast, TGCN uses GRU in the RNN kernel, which contains three types of weight parameters. Vertex features only need to be read three times instead of four times repeated under ReaDy-WF. Therefore, ReaDy offers a $1.95\times$ speedup for TGCN on average.

Interkernel Pipeline: Fig. 13 investigates the performance and off-chip memory access of ReaDy with and without interkernel pipelining. ReaDy-NP does not use pipelining by executing each kernel sequentially. As shown in Fig. 13(a), a pipelined execution in ReaDy can improve the performance by $1.96\times$ on average. The reasons are twofold. First, the GCN and RNN kernels work in parallel with the interkernel pipeline,

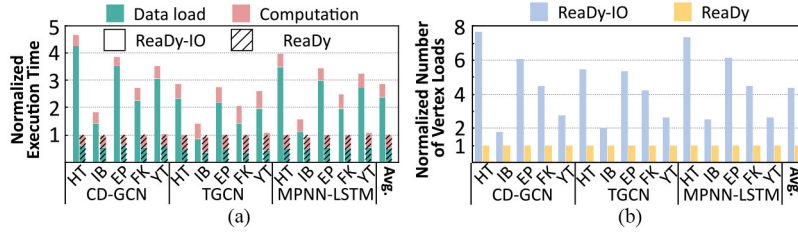


Fig. 11. (a) Execution times and (b) number of vertex data loads of ReaDy for the GCN kernel with different scheduling orders. All results are normalized to the redundancy-free scheduling.

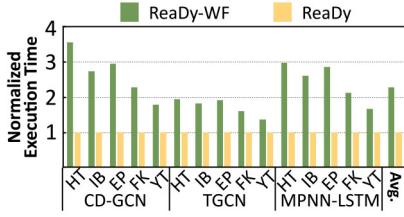


Fig. 12. Execution times of ReaDy for the RNN kernel with and without the weight matrix fusion. All results are normalized to ReaDy.

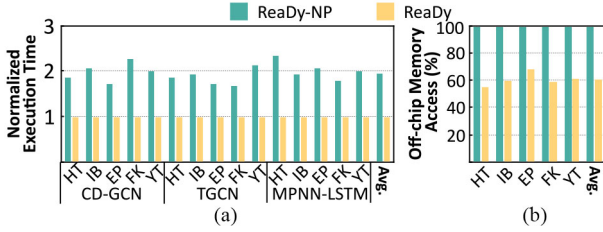


Fig. 13. Effects of the interkernel pipeline on (a) execution time and (b) off-chip memory access for ReaDy. All results are normalized to ReaDy.

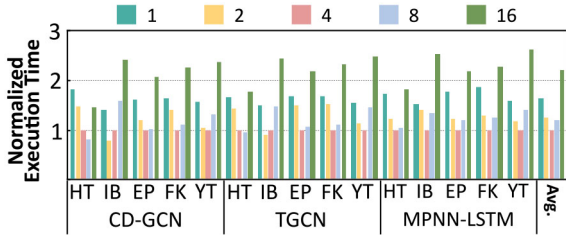


Fig. 14. Execution times of ReaDy with different values for the resource allocation parameter ϵ . All results are normalized to the default $\epsilon = 4$.

enabling hardware resources to be utilized adequately. Second, the interkernel pipeline avoids the intermediate data movement between on-chip PEs and off-chip memory, which occupies a considerable proportion of the overall execution time. Fig. 13(b) shows that the total number of off-chip memory accesses is significantly reduced to only 60.61% on average with the interkernel pipeline adopted.

D. Sensitivity Analysis

We analyze the sensitivity of ReaDy to the resource allocation parameter ϵ described in Section IV-C and different hidden feature dimensions.

Resource Allocation Parameter ϵ : Fig. 14 shows the performance of ReaDy with different values of ϵ . The aggregation phase of the GCN kernel may introduce the overhead

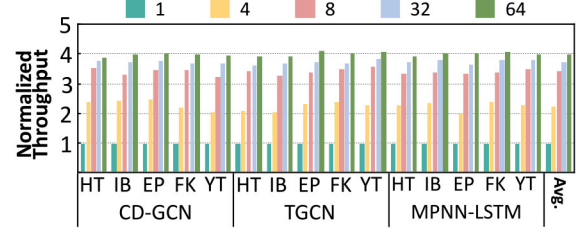


Fig. 15. Throughput of ReaDy with different vertex feature dimensions. All results are normalized to the feature dimension at 1.

for loading vertex data into the crossbar. Thus, ReaDy uses ϵ to accommodate such overhead as in (5). A suitable value for ϵ allows for load balancing across different kernels. Looking at $\epsilon = 4$, we see that increasing ϵ causes more resources to be allocated to the aggregation phase of the GCN kernel. Aggregation results cannot be processed by the downstream processing stages promptly, causing performance degradation. By decreasing ϵ , less aggregation results are produced, leaving not enough data for downstream processing, leading to stagnant resources. Overall, $\epsilon = 4$ seems to yield a sweep point for yielding good results. As for the problem of finding an optimal setting, we leave it as future work.

Feature Dimensions: Fig. 15 evaluates ReaDy with different feature dimensions. When the vertex feature dimension is small (under 8), the throughput of ReaDy increases as the number of dimensions increases. This is because large feature dimensions lead to more intravertex parallelism to be exploited by ReRAM crossbars. The throughput is saturated when the dimensionality exceeds the crossbar size (64), because the maximum ReRAM parallelism is fully exploited. In particular, the throughput of ReaDy reaches saturation more quickly when processing CD-GCN, mainly because it concatenates the input feature of the GCN kernel to the output, resulting in larger vertex feature dimensions than other models. Note that the feature dimension of DGCNs is usually much larger than 8. Thus, ReaDy allows the parallelism of the ReRAM crossbars to be fully exploited.

VI. RELATED WORK

There are lots of software frameworks [12], [52] for GCNs and RNNs that alleviate programming efforts while achieving high performance on CPUs and GPUs. However, they concentrate only on either GCNs or RNNs but hardly work well for DGCNs consisting of both GCNs and RNNs. Therefore, there are a few libraries and systems specifically optimized for DGCNs. PyGT [41] provides an optimized model library built on top of widely used geometric deep learning libraries from the PyTorch ecosystem. IBM Research has also developed a

scalable implementation for DGCNs models on distributed multinode systems [7]. Unfortunately, irregular graph structure imposes intrinsic random accesses in GCNs, and data dependencies incur data conflicts in RNNs, limiting the benefit potential achieved on traditional platforms.

With the emergence of GCN and RNN applications, specialized architectures are proposed to accelerate these workloads.

For GCNs, HyGCN [50] represents a hybrid architecture with specialized modules for aggregation and combination, respectively. AWB-GCN [13] introduces a workload autotuning technique to address workload imbalance issues caused by the power-law distribution of real-world graphs. GCNAX [29] reconfigures the dataflow of GCNs to reduce data movements. Rubik [9] and I-GCN [14] improve data locality by offline and online graph restructuring algorithms, respectively. ReFlip [22] represents a unified architecture for accelerating GCNs by leveraging ReRAM crossbars. In contrast to these prior efforts focusing on static graphs, ReaDY is designed to support GCN models on dynamic graphs. ReGraphX [4] proposes to use a 3-D NoC for heterogeneous ReRAM crossbars to improve communication efficiency. ReMaGN [5] incorporates the reduced-precision representation to reduce data traffic amount. DARE [3] further reduces the data traffic amount using the DropLayer mechanism. From the perspective of computation, ReaDY performs both the GCN and RNN kernels efficiently. From the perspective of data, ReaDY is compatible with the dynamic variation of graph data during execution.

For RNNs, ESE [17] includes a scheduler that encodes and partitions the RNN models to multiple PEs for parallelizing the dataflow of RNNs on FPGA. E-PUR [43] introduces a technique to improve the temporal locality of the memory accesses for fetching the synaptic weights. Silfa *et al.* [44] further propose a fuzzy memoization scheme to reuse previously computed results stored in a memoization buffer and implement it on top of E-PUR. ERA-LSTM [16] analyzed the dataflow of RNNs and presents an interlayer pipeline technique on the ReRAM-based architecture. Echo [55] features a compiler-based optimization scheme to automatically and transparently reduce the memory footprint of RNNs. Unlike all prior efforts focusing on optimizing data dependencies between layers, ReaDY exploits the unique intervertex parallelism of DGCNs to process the RNN kernel layer by layer to fundamentally decouple the complex data dependencies between layers.

In dynamic graph processing, most existing research efforts fall into two categories. The first category improves the efficiency of graph computation under dynamic graph processing scenes through delta-accumulative computation, which adopts the intermediate (rather than initial) vertex state to accelerate the convergence of graph computation [38], [39]. GraphPulse [38] adopts an event-driven model to enable computation on the updated graph. JetStream [39] further supports edge additions and deletion for accumulative and monotonic graph algorithms simultaneously. However, DGCNs behave not only like the memory-intensive graph traversal-based execution pattern but also the computation-intensive neural networks-based execution pattern. Thus, these specialized architectures cannot efficiently perform DGCNs. The second category supports high-throughput graph updates via specialized data management [6], [48]. GraSU [48] offers a graph-structured update library with differential data management on FPGA. Basak *et al.* [6] presented an input-aware batch reordering scheme and design a specialized unit to improve

the graph update efficiency. In contrast to these studies focusing on graph update, ReaDY mainly considers scenarios where graph snapshots are available and is thus orthogonal to these existing high-throughput graph update techniques.

Since the key computation pattern in DGCNs is matrix multiplication, another related topic is matrix multiplication accelerators [18], [35], [46], [51], [53]. OuterSpace [35] and ExTensor [18] apply outer and inner products to reuse input data and reduce partial data traffic, respectively. SpArch [53] optimizes partial data under the outer product manner through a condensed matrix representation and a Huffman tree scheduler. MatRaptor [46] and Gamma [51] adopt the row-wise product to improve the data locality for both input and partial data. However, since real-world graphs are extremely sparse, these works typically handle general sparse matrices, and may not process graph data well. Furthermore, DGCNs can be formalized as matrix chain multiplication, yielding more intermediate data. Therefore, existing approaches may work well for a single matrix multiplication, but not for those likely to be processed by DGCNs.

VII. CONCLUSION

In this article, we propose the first DGCN accelerator, ReaDY, with an integrated architecture that allows running the GCN and RNN kernels uniformly on ReRAM crossbars. To fully exploit the hardware potential of the integrated architecture, ReaDY is designed with three technical innovations. First, ReaDY adopts a redundancy-free scheduling mechanism to alleviate dynamic irregularity for the GCN kernel. Second, ReaDY's locality-aware dataflow execution is designed to maximize the intervertex data reuse for the RNN kernel. Finally, ReaDY includes an interkernel pipeline to maximize overall efficiency by eliminating the off-chip memory access to excessive intermediate data. ReaDY is shown to significantly outperform the state-of-the-art CPU, GPU, and accelerator solutions in terms of both performance and energy efficiency.

REFERENCES

- [1] "Intel Xeon E5-2680 v4." [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/91754/intel-xeon-processor-e52680-v4-35m-cache-2-40-ghz/specifications.html>
- [2] "NVIDIA system management interface." 2012. [Online]. Available: <https://developer.nvidia.com/nvidia-system-management-interface/>
- [3] A. I. Arka, B. K. Joardar, J. R. Doppa, P. P. Pande, and K. Chakrabarty, "DARE: DropLayer-aware manycore ReRAM architecture for training graph neural networks," in *Proc. Int. Conf. Comput.-Aided Des. (ICCAD)*, 2021, pp. 1–9.
- [4] A. I. Arka, J. R. Doppa, P. P. Pande, B. K. Joardar, and K. Chakrabarty, "ReGraphX: NoC-enabled 3D heterogeneous ReRAM architecture for training graph neural networks," in *Proc. Des. Autom. Test Europe Conf. Exhibit. (DATE)*, 2021, pp. 1667–1672.
- [5] A. I. Arka, B. K. Joardar, J. R. Doppa, P. P. Pande, and K. Chakrabarty, "Performance and accuracy tradeoffs for training graph neural networks on ReRAM-based architectures," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 10, pp. 1743–1756, Oct. 2021.
- [6] A. Basak *et al.*, "Improving streaming graph processing performance using input knowledge," in *Proc. Annu. Int. Symp. Microarchit. (MICRO)*, 2021, pp. 1036–1050.
- [7] V. T. Chakaravarthy, S. S. Pandian, S. Raje, Y. Sabharwal, T. Suzumura, and S. Ubaru, "Efficient scaling of dynamic graph neural networks," in *Proc. Conf. High Perform. Comput. Netw. Storage Anal. (SC)*, 2021, pp. 1–15.
- [8] J. Chen, G. Lin, J. Chen, and Y. Wang, "Towards efficient allocation of graph convolutional networks on hybrid computation-in-memory architecture," *Sci. China Inf. Sci.*, vol. 64, no. 6, pp. 1–14, 2021.
- [9] X. Chen *et al.*, "Rubik: A hierarchical architecture for efficient graph neural network training," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 4, pp. 936–949, Apr. 2022.

- [10] P. Chi *et al.*, "PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory," in *Proc. Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 27–39.
- [11] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder–decoder approaches," in *Proc. 8th Workshop Syntax Semantics Struct. Stat. Transl. (SSST)*, 2014, pp. 103–111.
- [12] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch geometric," 2019, *arXiv:1903.02428*.
- [13] T. Geng *et al.*, "AWB-GCN: A graph convolutional network accelerator with runtime workload rebalancing," in *Proc. Annu. Int. Symp. Microarchit. (MICRO)*, 2020, pp. 922–936.
- [14] T. Geng *et al.*, "I-GCN: A graph convolutional network accelerator with runtime locality enhancement through islandization," in *Proc. Annu. Int. Symp. Microarchit. (MICRO)*, 2021, pp. 1051–1063.
- [15] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 70, 2017, pp. 1263–1272.
- [16] J. Han, H. Liu, M. Wang, Z. Li, and Y. Zhang, "ERA-LSTM: An efficient ReRAM-based architecture for long short-term memory," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 6, pp. 1328–1342, Jun. 2020.
- [17] S. Han *et al.*, "ESE: Efficient speech recognition engine with sparse LSTM on FPGA," in *Proc. Symp. Field-Program. Gate Arrays (FPGA)*, 2017, pp. 75–84.
- [18] K. Hegde *et al.*, "ExTensor: An accelerator for sparse tensor algebra," in *Proc. Int. Symp. Microarchit. (MICRO)*, 2019, pp. 319–333.
- [19] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [20] M. Hu *et al.*, "Dot-product engine for neuromorphic computing: Programming 1T1M crossbar to accelerate matrix-vector multiplication," in *Proc. Annu. Des. Autom. Conf. (DAC)*, 2016, pp. 1–6.
- [21] Y. Huang, L. Zheng, X. Liao, H. Jin, P. Yao, and C. Gui, "RAGra: Leveraging monolithic 3D ReRAM for massively-parallel graph processing," in *Proc. Des. Autom. Test Europe Conf. Exhibit. (DATE)*, 2019, pp. 1273–1276.
- [22] Y. Huang *et al.*, "Accelerating graph convolutional networks using crossbar-based processing-in-memory architectures," in *Proc. Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2022, pp. 1029–1042.
- [23] Y. Huang *et al.*, "A heterogeneous PIM hardware-software co-design for energy-efficient graph processing," in *Proc. Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2020, pp. 684–695.
- [24] B. E. Jonsson, "A survey of A/D-converter performance evolution," in *Proc. Int. Conf. Electron. Circuits Syst. (ICECS)*, 2010, pp. 766–769.
- [25] S. M. Kazemi, R. Goel, K. Jain, I. Kobayev, A. Sethi, and P. Poupart, "Representation learning for dynamic graphs: A survey," *J. Mach. Learn. Res.*, vol. 21, pp. 2648–2720, Jan. 2020.
- [26] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible DRAM simulator," *IEEE Comput. Archit. Lett.*, vol. 15, no. 1, pp. 45–49, Jan.–Jun. 2016.
- [27] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 1–14.
- [28] "CACTI." HP Labs. 2016. [Online]. Available: <http://www.hpl.hp.com/research/cacti/>
- [29] J. Li, A. Louri, A. Karanth, and R. C. Bunescu, "GCNAX: A flexible and energy-efficient accelerator for graph convolutional neural networks," in *Proc. Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2021, pp. 775–788.
- [30] W. Li, P. Xu, Y. Zhao, H. Li, Y. Xie, and Y. Lin, "TIMELY: Pushing data movements and interfaces in PIM accelerators towards local and in time domain," in *Proc. Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 832–845.
- [31] F. Manessi, A. Rozza, and M. Manzo, "Dynamic graph convolutional networks," *Pattern Recognit.*, vol. 97, pp. 1–18, Jan. 2020.
- [32] A. Mukkara, N. Beckmann, M. Abeydeera, X. Ma, and D. Sánchez, "Exploiting locality in graph analytics through hardware-accelerated traversal scheduling," in *Proc. Annu. Int. Symp. Microarchit. (MICRO)*, 2018, pp. 1–14.
- [33] D. Niu, C. Xu, N. Muralimanohar, N. P. Jouppi, and Y. Xie, "Design of cross-point metal-oxide ReRAM emphasizing reliability and cost," in *Proc. Int. Conf. Comput.-Aided Des. (ICCAD)*, 2013, pp. 17–23.
- [34] M. O'Connor, "Highlights of the high-bandwidth memory (HBM) standard," in *Proc. Memory Forum Workshop (MFW)*, 2014, pp. 1–25.
- [35] S. Pal *et al.*, "OuterSPACE: An outer product based sparse matrix multiplication accelerator," in *Proc. Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2018, pp. 724–736.
- [36] G. Panagopoulos, G. Nikolentzos, and M. Vazirgiannis, "Transfer graph neural networks for pandemic forecasting," in *Proc. Conf. Artif. Intell. (AAAI)*, 2021, pp. 4838–4845.
- [37] X. Qian, "Graph processing and machine learning architectures with emerging memory technologies: A survey," *Sci. China Inf. Sci.*, vol. 64, no. 6, pp. 1–25, 2021.
- [38] S. Rahman, N. B. Abu-Ghazaleh, and R. Gupta, "GraphPulse: An event-driven hardware accelerator for asynchronous graph processing," in *Proc. Annu. Int. Symp. Microarchit. (MICRO)*, 2020, pp. 908–921.
- [39] S. Rahman, M. Afarin, N. B. Abu-Ghazaleh, and R. Gupta, "JetStream: Graph analytics on streaming data with event-driven hardware accelerator," in *Proc. Annu. Int. Symp. Microarchit. (MICRO)*, 2021, pp. 1091–1105.
- [40] R. A. Rossi and N. K. Ahmed, "The network data repository with interactive graph analytics and visualization," in *Proc. Conf. Artif. Intell. (AAAI)*, 2015, pp. 4292–4293.
- [41] B. Rozemberczki *et al.*, "PyTorch geometric temporal: Spatiotemporal signal processing with neural machine learning models," in *Proc. Int. Conf. Inf. Knowl. Manag. (CIKM)*, 2021, pp. 4564–4573.
- [42] A. Shafiee *et al.*, "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 14–26.
- [43] F. Silfa, G. Dot, J. Arnau, and A. González, "E-PUR: An energy-efficient processing unit for recurrent neural networks," in *Proc. Int. Conf. Parallel Archit. Compilation Techn. (PACT)*, 2018, pp. 1–12.
- [44] F. Silfa, G. Dot, J. Arnau, and A. González, "Neuron-level fuzzy mem- oization in RNNs," in *Proc. Annu. Int. Symp. Microarchit. (MICRO)*, 2019, pp. 782–793.
- [45] L. Song, Y. Zhuo, X. Qian, H. H. Li, and Y. Chen, "GraphR: Accelerating graph processing using ReRAM," in *Proc. Symp. High Perform. Comput. Archit. (HPCA)*, 2018, pp. 531–543.
- [46] N. K. Srivastava, H. Jin, J. Liu, D. H. Albonesi, and Z. Zhang, "MatRaptor: A sparse-sparse matrix multiplication accelerator based on row-wise product," in *Proc. Annu. Int. Symp. Microarchit. (MICRO)*, 2020, pp. 766–780.
- [47] T. Stutzle and H. Hoos, "MAX–MIN ant system and local search for the Traveling salesman problem," in *Proc. Int. Conf. Evol. Comput. (ICEC)*, 1997, pp. 309–314.
- [48] Q. Wang *et al.*, "GraSU: A fast graph update library for FPGA-based dynamic graph processing," in *Proc. Symp. Field-Program. Gate Arrays (FPGA)*, 2021, pp. 149–159.
- [49] Q. Wang *et al.*, "Hardware-accelerated hypergraph processing with chain-driven scheduling," in *Proc. Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2022, pp. 184–198.
- [50] M. Yan *et al.*, "HyGCN: A GCN accelerator with hybrid architecture," in *Proc. Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2020, pp. 15–29.
- [51] G. Zhang, N. Attaluri, J. S. Emer, and D. Sánchez, "Gamma: Leveraging Gustavson's algorithm to accelerate sparse matrix multiplication," in *Proc. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2021, pp. 687–701.
- [52] M. Zhang, S. Rajbhandari, W. Wang, and Y. He, "DeepCPU: Serving RNN-based deep learning models 10x faster," in *Proc. USENIX Annu. Tech. Conf. (ATC)*, 2018, pp. 951–965.
- [53] Z. Zhang, H. Wang, S. Han, and W. J. Dally, "SpArch: Efficient architecture for sparse matrix multiplication," in *Proc. Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2020, pp. 261–274.
- [54] L. Zhao *et al.*, "T-GCN: A temporal graph convolutional network for traffic prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 9, pp. 3848–3858, Sep. 2020.
- [55] B. Zheng, N. Vijaykumar, and G. Pekhimenko, "Echo: Compiler-based GPU memory footprint reduction for LSTM RNN training," in *Proc. Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 1089–1102.